

AI PROJECT · SPRING 2026

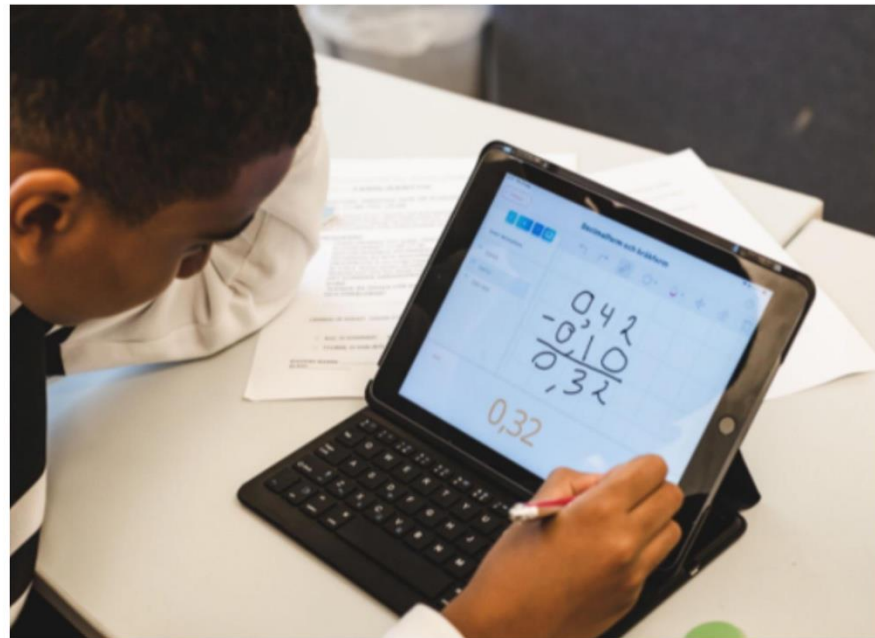
Draw & Solve

Handwritten Math Expression Recognition

Name	Student ID	Task
Matti Hannes Vincent Lehmann	T14H06309	Presentation Slides
David Axel Spickenheuer	T14H06329	Data Preparation & Augmentation
Jonas Schenke	T14H06307	CNN Training & Evaluation
Philip Nguyen	91499153X	Presentation Slides
Staniya Thomas	T14H06310	Symbol Recognition Pipeline
Abhirup Sain	T14H06318	Streamlit UI & Integration

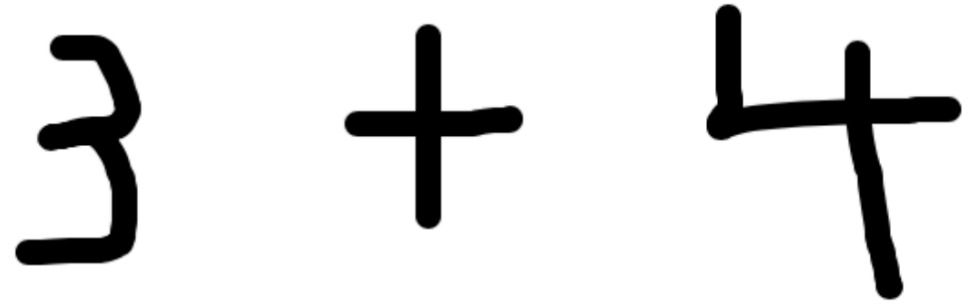
- **Handwritten maths** on tablets and touch screens is increasingly common (online exams, digital whiteboards, teaching apps)
- **Manual typing** of expressions into a calculator or CAS is slow and error-prone
- **Goal:** draw an expression once and obtain an instant, reliable numerical result

Why now? Tablet adoption in education has grown rapidly — a seamless, pen-first math solver removes a major friction point for students and teachers.



- Handwriting varies widely per writer
- Operators share features with digits (e.g. $\times$ vs x, $-$ vs $\div$)
- Multi-stroke digits confuse naive segmenters

Goal: Draw a math expression on screen $\rightarrow$ get the result instantly.



1. Draw an expression on a browser canvas
2. OpenCV segments individual symbols
3. CNN classifies each symbol (14 classes)
4. Safe AST evaluator computes the result

Digits

0 · 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9

Operators

+ · − · × · ÷

Detected symbols

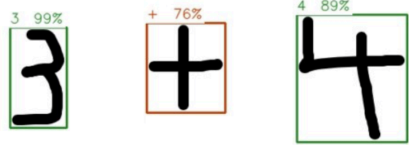
3 + 4

Expression

3+4

Result

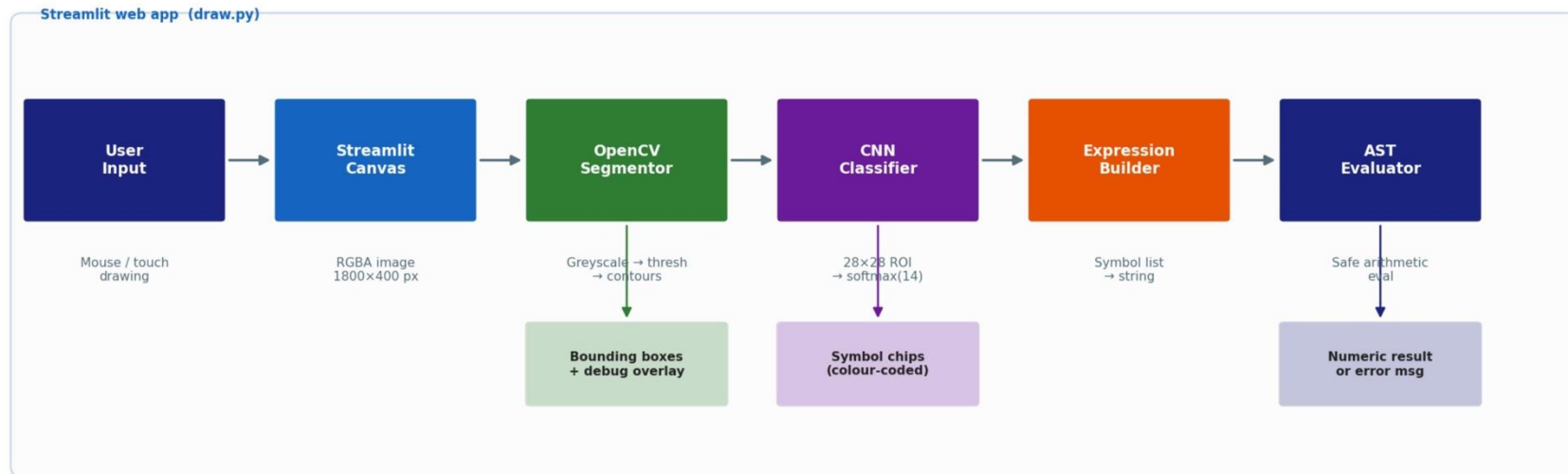
7



Detected symbols (green = digit · teal = operator · orange = uncertain)

The image shows a digital interface for solving a math problem. At the top, it displays 'Detected symbols' as '3 + 4' in colored boxes (green for digits, teal for operators). Below this, the 'Expression' is '3+4' and the 'Result' is '7'. A large white box contains a hand-drawn version of the expression '3+4'. Each symbol in the drawing is enclosed in a colored bounding box with a confidence score: '3' (green, 99%), '+' (orange, 76%), and '4' (green, 89%). A legend at the bottom explains the color coding: green for digits, teal for operators, and orange for uncertain symbols.

Software Architecture — Draw & Solve



Model: outputs/best_model.keras · Classes: outputs/class_names.json

Frontend

Streamlit · HTML5 Canvas

Backend

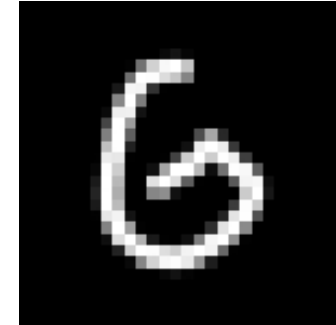
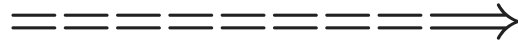
Python · OpenCV · TensorFlow

Model

Keras · 14-class CNN



How to get from the drawing to the CNN-input?



First step: Binarisation

- Canvas: white (255), ink ≈ 0
- THRESH_BINARY_INV: ink $\rightarrow 255$, bg $\rightarrow 0$
- 3×3 dilation closes gaps between strokes
- findContours detects white regions on black

6 ÷ 3

6 ÷ 3

6 ÷ 3

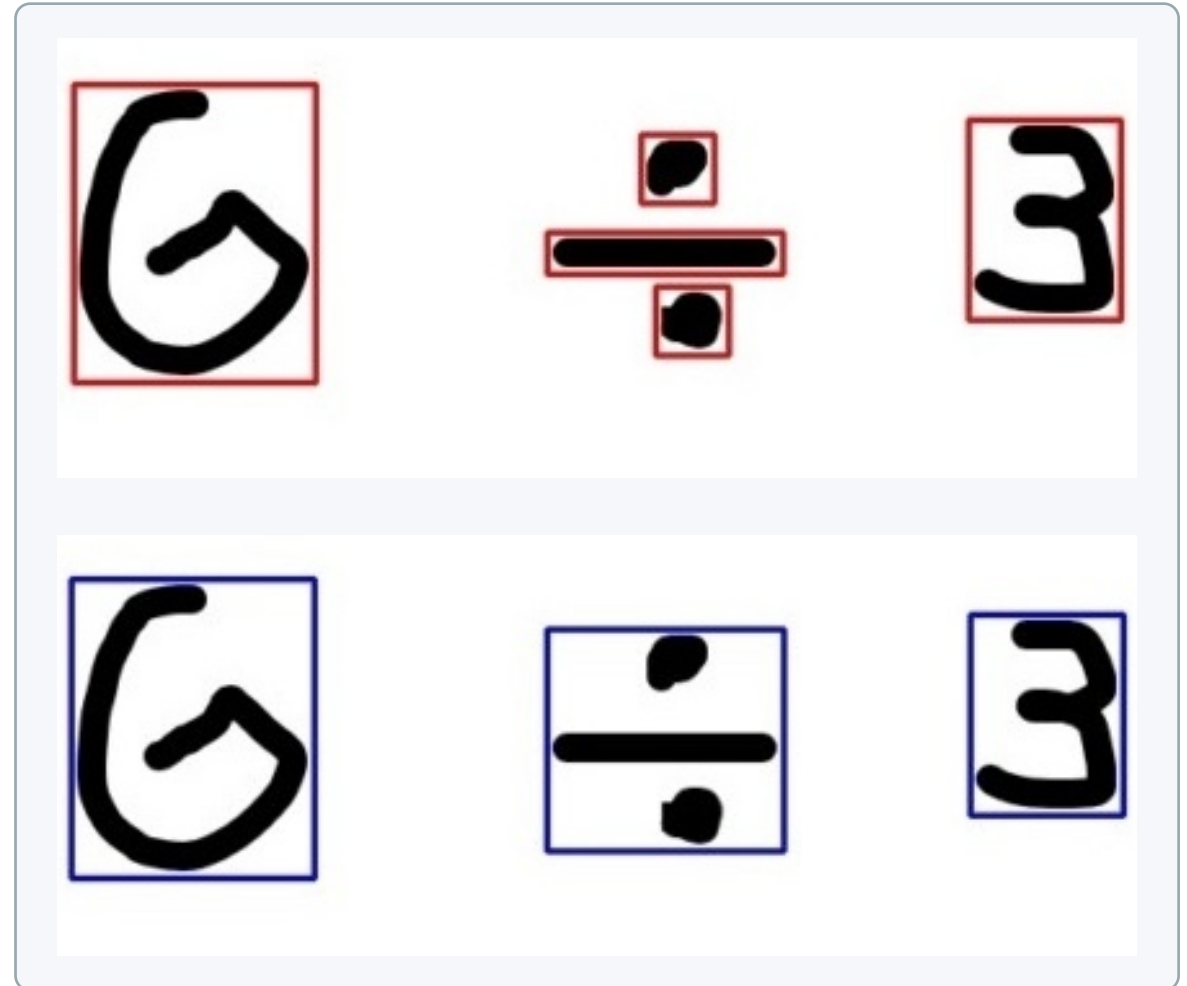
Bounding boxes

- Outermost contours only (RETR_EXTERNAL)
- Box area $< 30 \text{ px}^2$ discarded (noise filter)
- Sort all boxes left $\rightarrow$ right by x-coordinate

Problem: signs and digits can be multi-stroke ($\div$, 5, 7)
 $\rightarrow$ each stroke gets its own box

Solution: merge boxes with gap $\leq 20 \text{ px}$

- Iterate boxes left $\rightarrow$ right
- If next box starts within 20 px, expand current box



Final Step: Conversion of boxed signs to CNN input-format

1. Cropping of Box-content
2. Add padding
3. Downscaling of image to fit in 28 x 28 format
4. centering + additional padding



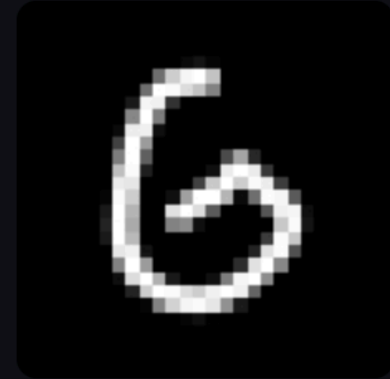
1. Cropped ROI



2. Padding Added



3. Scaled Down



4. Centered in 28x28

Architecture · Training · Evaluation



14

Classes

177K

Params

64192

Training Samples

Problem: Imbalance between classes

Source

Digits 0-9

Operators $+-\times\div$

Detail

MNIST (LeCun et al.)

Sagyamthapa (Kaggle)

14:1 Ratio between MNIST-digits and operators

Solution: AUGMENTATION PIPELINE



Random Rotation

$\pm 8\%$ (28.8°)

$\pm 8\%$ (28.8°)



Random Translation

$\pm 8\%$ H & V

$\pm 8\%$ H & V



Random Zoom

$\pm 10\%$

$\pm 10\%$



Elastic Transform

$\alpha=8$ $\sigma=4$ $p=0.7$

$\alpha=8$ $\sigma=4$ $p=0.7$

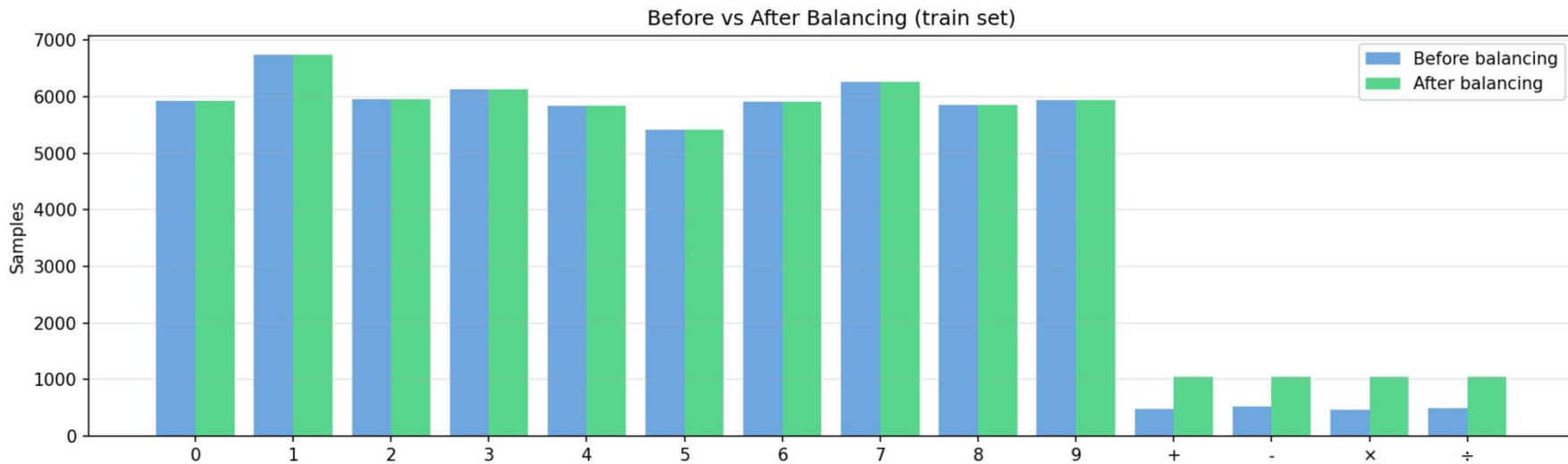


Pixel Normalize

$\div 255 \rightarrow [0,1]$

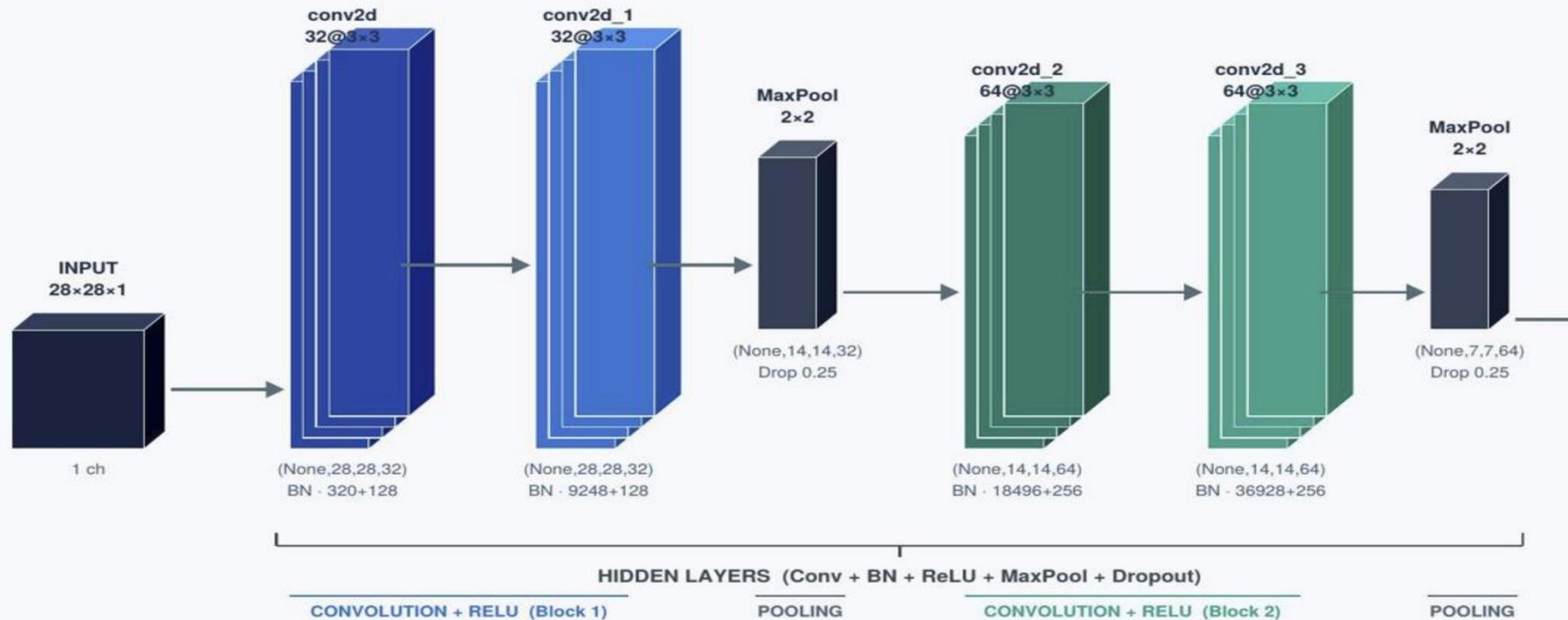
$\div 255 \rightarrow [0,1]$

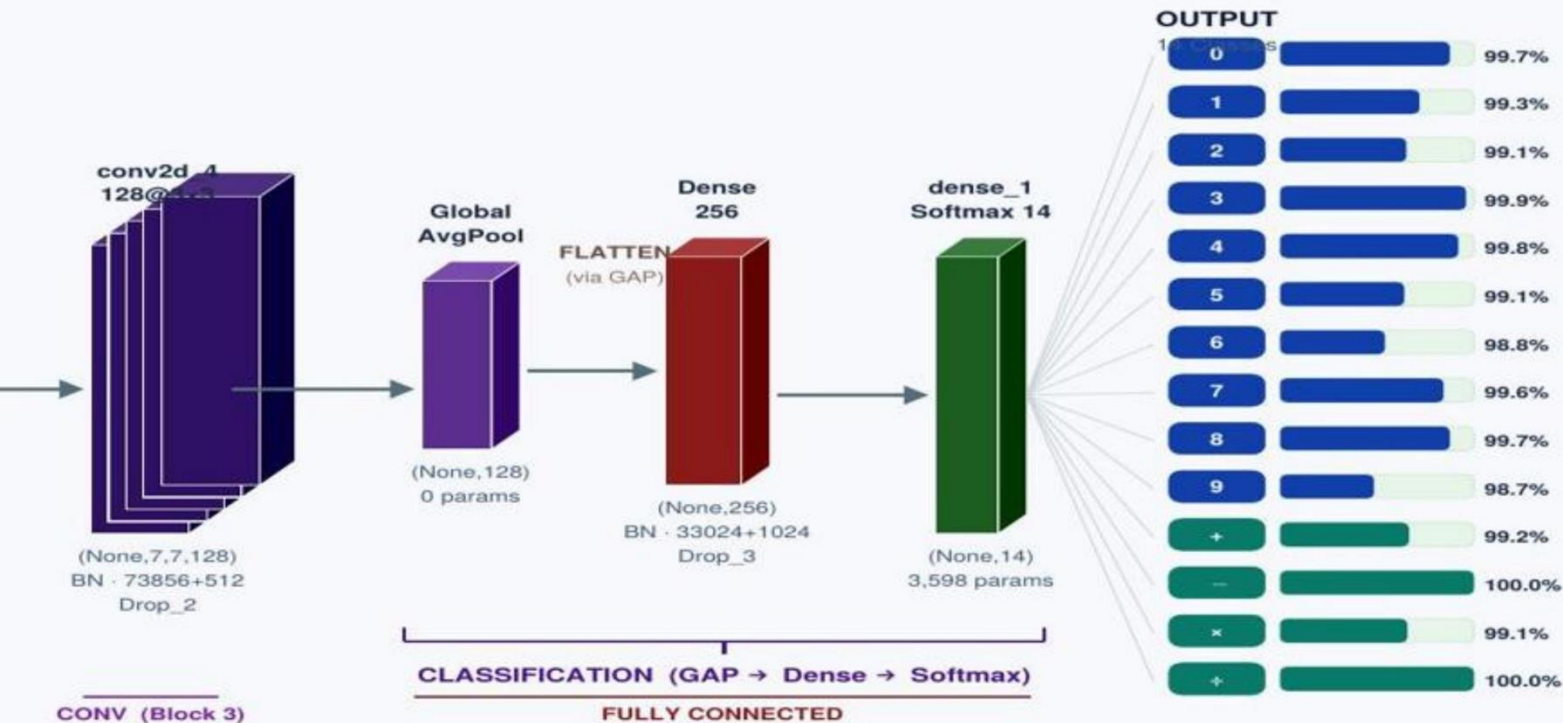
- Digits: 5 000 – 6 700 samples each
- Operators: 1 048 samples each → **6:1 ratio**
- Without fix: model predicts digits almost always



Convolution Neural Network (CNN) Architecture

16





Batch Normalization Rescales layer outputs to mean zero and unit variance. Accelerates training by up to 3× and provides mild regularization.

MaxPooling (2×2) Downsamples spatial dimensions to reduce computation costs while making detected features invariant to small translational shifts.

Dropout Regularization Randomly silences 25–50% of neurons during training. This prevents memorization and forces robust, distributed learning.

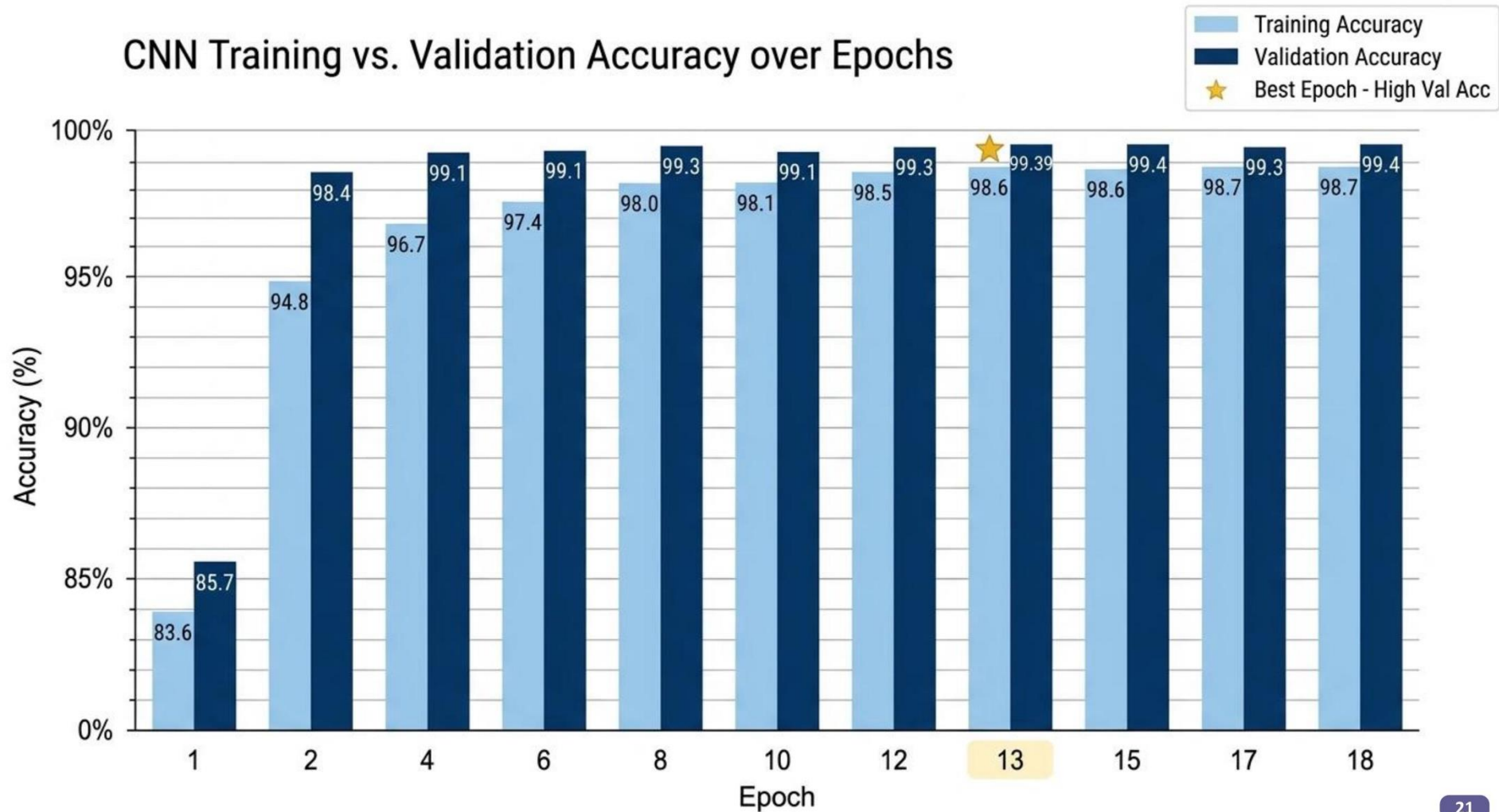
GlobalAveragePooling Replaces the Flatten layer by averaging feature maps into a compact vector, dramatically reducing model parameter count.

Optimizer	AdamW lr=1e-3 wd=1e-4
Loss	Categorical Cross-Entropy
Batch Size	128
Epochs	20 (early stop @ 18)
Best Epoch	Epoch 13
Callbacks	EarlyStopping(patience=5) + ReduceLROnPlateau

LR SCHEDULE



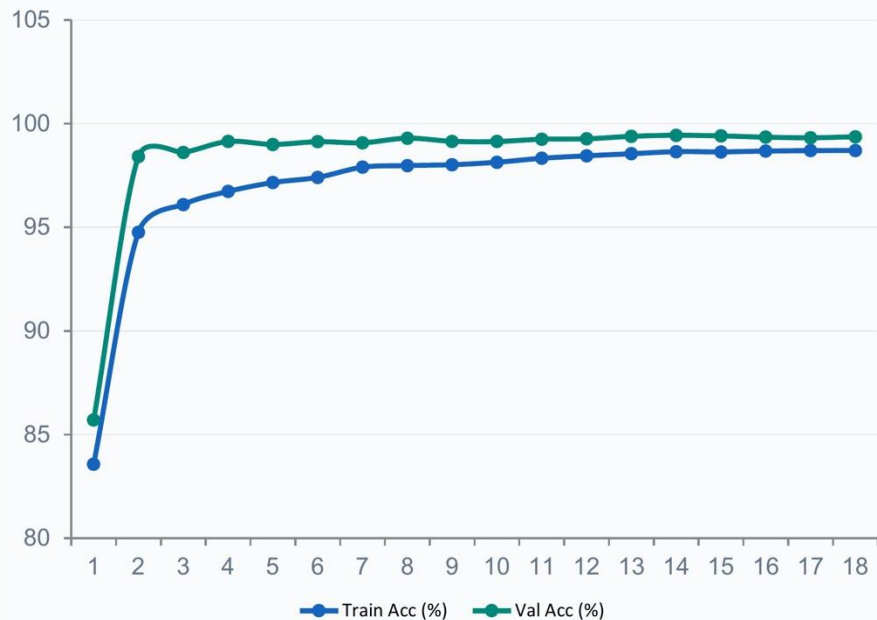
CNN Training vs. Validation Accuracy over Epochs



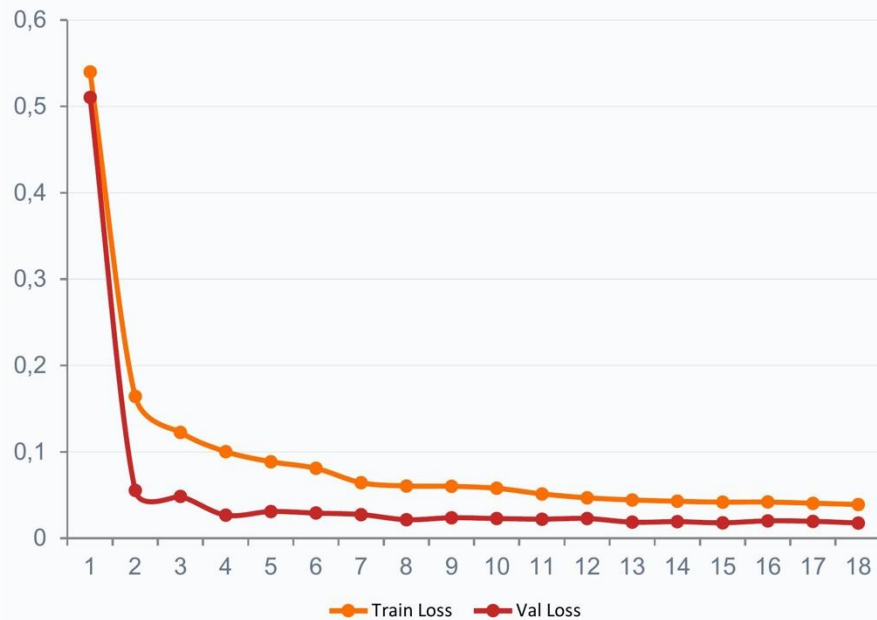
Training Progress : Accuracy & Loss Curves

22

Accuracy per Epoch



Loss per Epoch

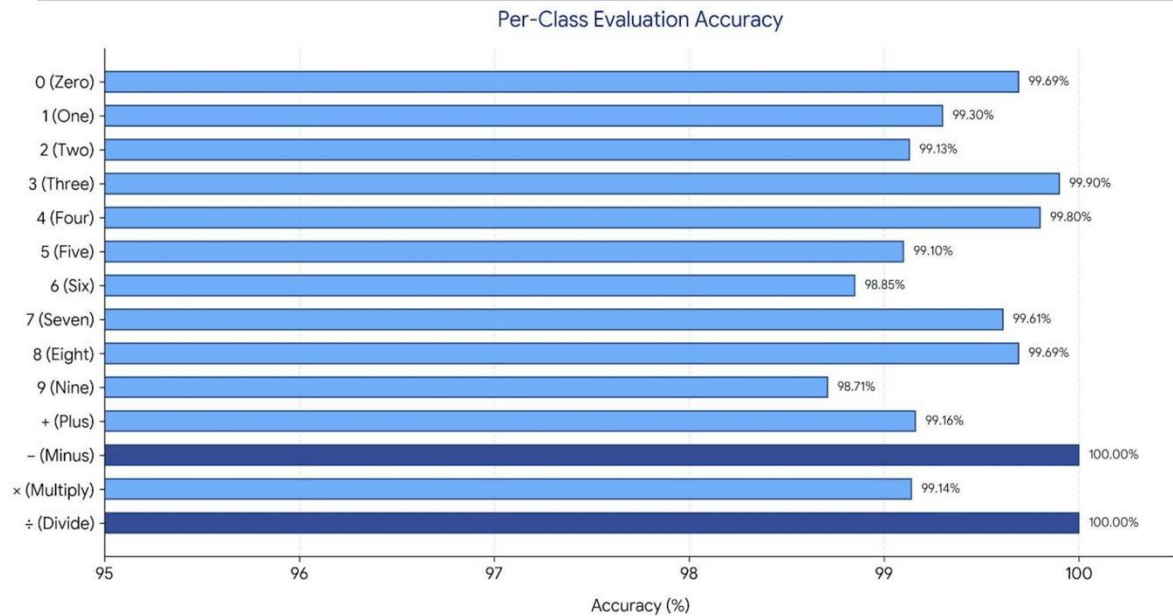


Fast convergence: val acc jumped from 85.7% → 98.4% in just 2 epochs

Best epoch: 13 (val acc 99.39%, val loss 0.0186)

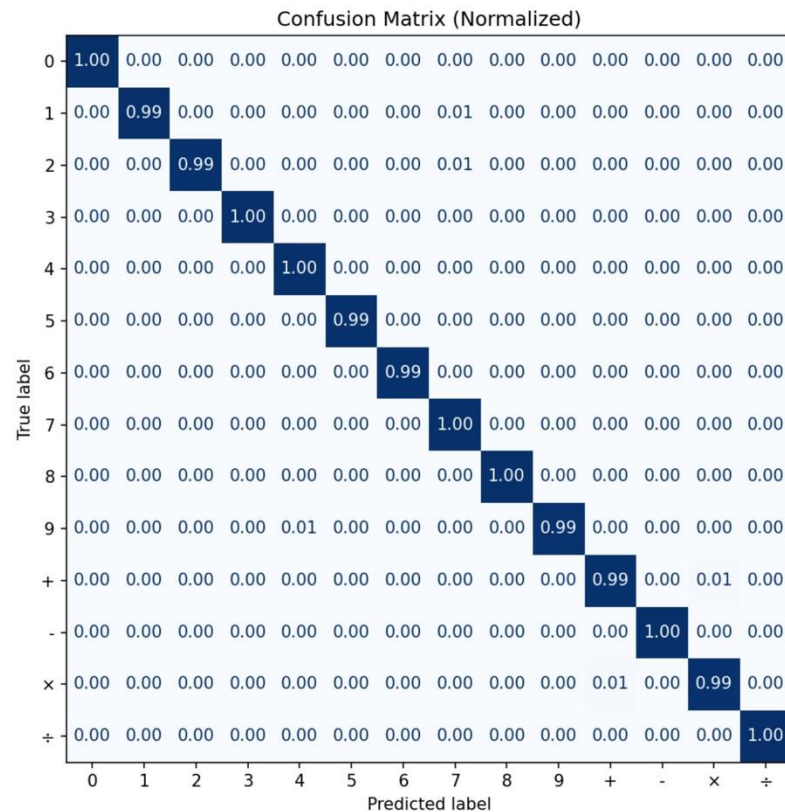
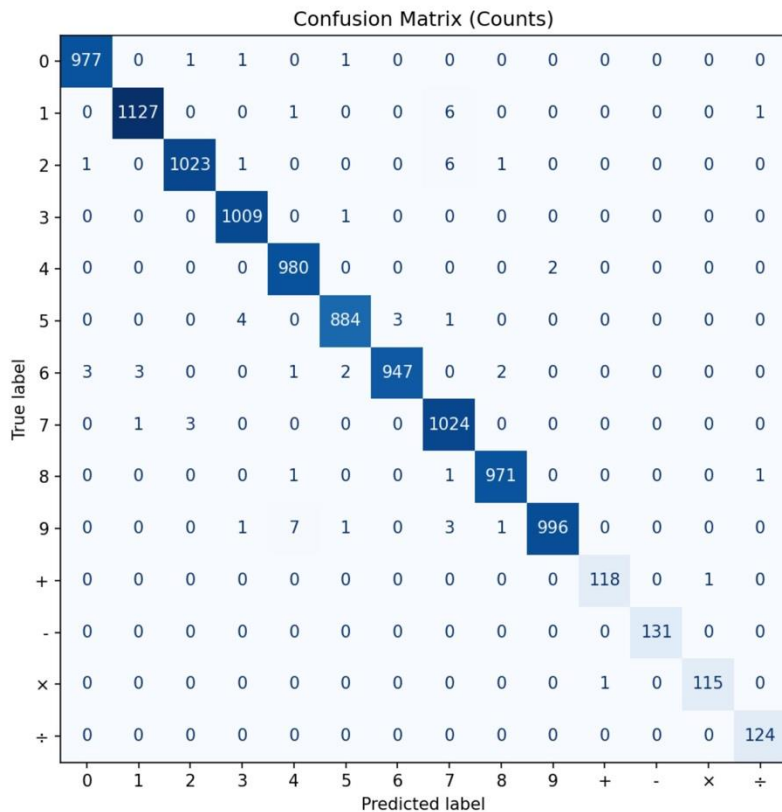
Early stopping triggered at epoch 18, no overfitting observed

ReduceLROnPlateau halved LR at epochs 6 / 10 / 12 / 17



Confusion Matrix

23



Top confusion pairs (test set, 59 total errors)

True	Predicted	Count	Why?
4	9	5	Open loop vs. closed
5	3	5	Similar curvature
9	4	4	Open loop vs. stem
7	2	4	Diagonal bar confusion
×	7	3	Cross $\approx$ slanted stroke

Key observations

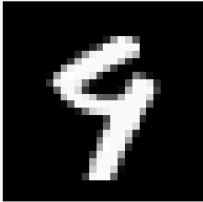
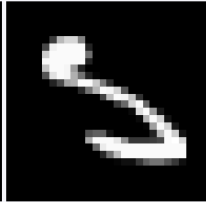
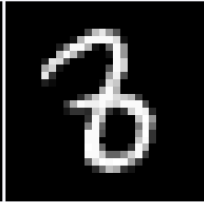
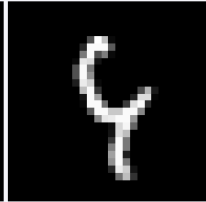
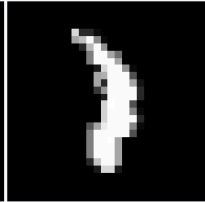
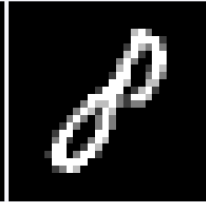
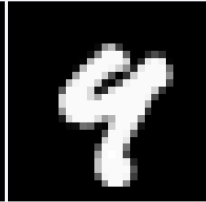
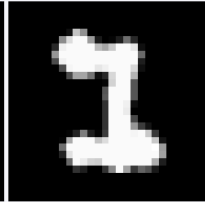
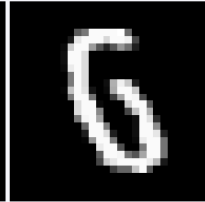
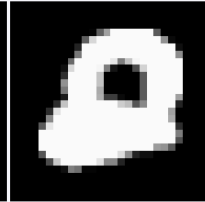
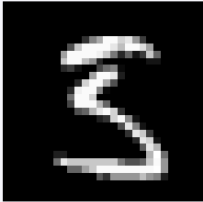
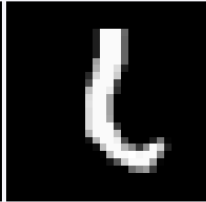
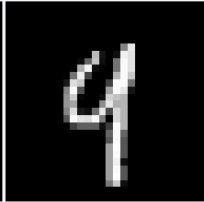
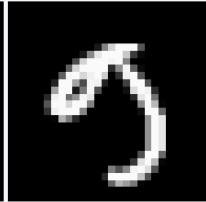
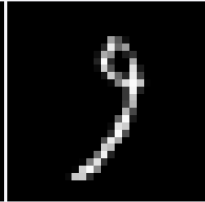
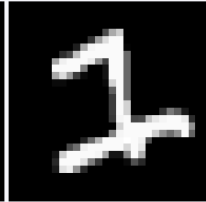
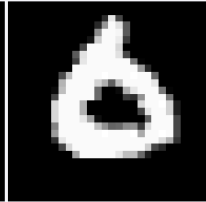
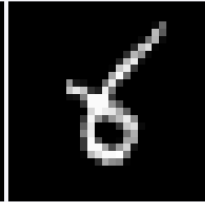
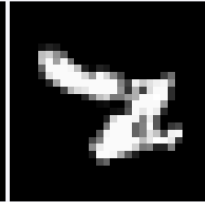
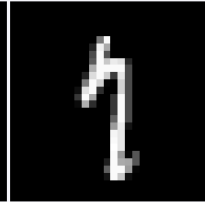
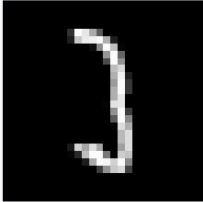
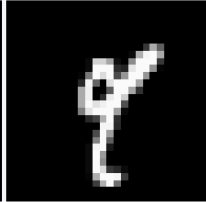
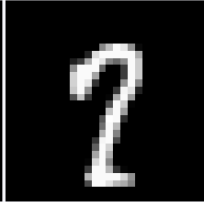
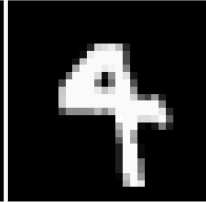
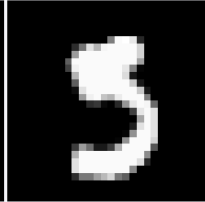
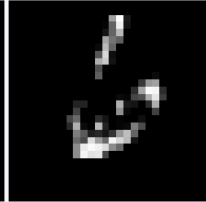
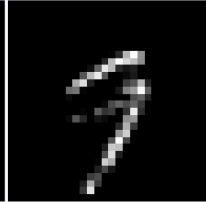
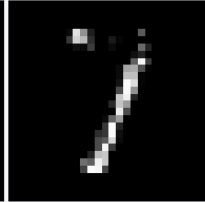
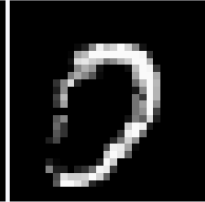
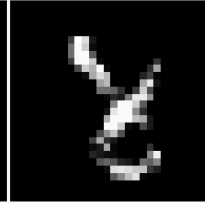
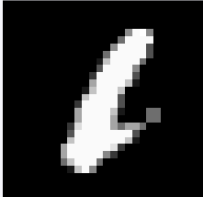
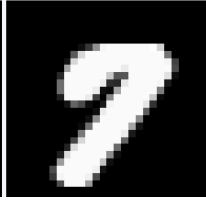
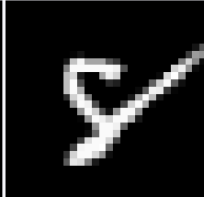
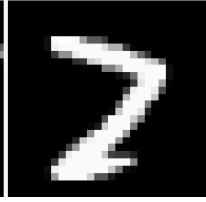
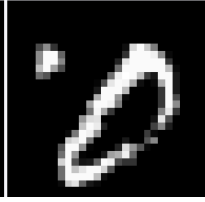
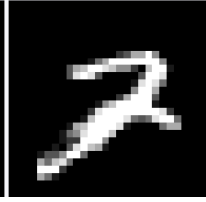
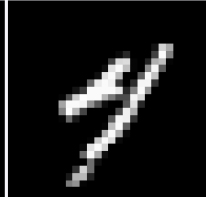
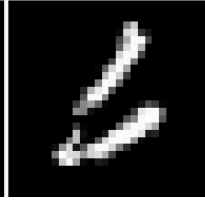
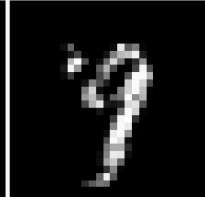
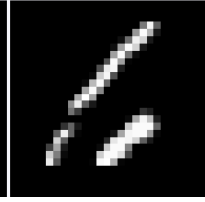
- $\times$ is the weakest class: 94.8% accuracy (6 errors in 116)
- $-$ and $\div$: 100% accuracy on the test set
- Most errors are **digit-to-digit**, not cross-type
- Worst pairs are visually near-identical shapes

With only 1 048 operator samples, $\times$ is most sensitive to style variation — being confused as a slanted 7.

Inference & assembly

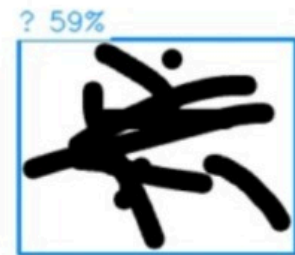
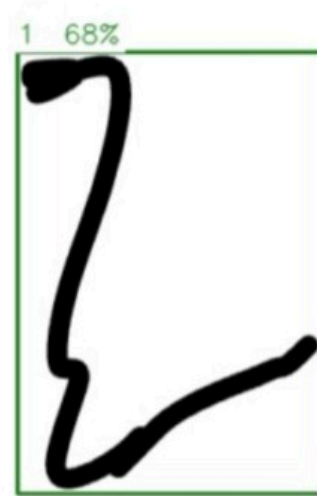
- Confidence $< 0.60 \rightarrow$ label as ?
- Sort symbols by $x \rightarrow$ expression string
- AST evaluator: only $+$, $-$, $\times$, $\div$ permitted (no `eval()`)

Failure Cases — All 59 Misclassified Symbols

✓9 → X4 70%	✓5 → X3 68%	✓8 → X3 54%	✓9 → X4 97%	✓1 → X2 53%	✓8 → X0 55%	✓4 → X9 97%	✓1 → X3 47%	✓6 → X0 58%	✓2 → X0 67%
									
✓5 → X3 69%	✓6 → X1 90%	✓4 → X9 91%	✓9 → X5 61%	✓9 → X7 67%	✓1 → X2 51%	✓6 → X0 87%	✓6 → X8 86%	✓2 → X7 86%	✓1 → X7 81%
									
✓3 → X2 76%	✓9 → X4 45%	✓7 → X2 72%	✓4 → X9 41%	✓5 → X3 50%	✓6 → X÷ 59%	✓9 → X7 60%	✓7 → X1 99%	✓0 → X7 64%	✓8 → X2 45%
									
✓1 → X6 71%	✓7 → X2 54%	✓8 → X4 98%	✓7 → X2 87%	✓0 → X2 71%	✓2 → X7 91%	✓4 → X7 88%	✓6 → X1 91%	✓9 → X7 48%	✓6 → X1 99%
									

Solver needs single characters

- probability vector gets converted to char
- confidence > 60% → decision
- otherwise return “?”



Solver

- chars merged to string expression
- expression solved with AST evaluator

99.44% Test accuracy

14 Unified classes

178K Parameters

< 3 min Training time

What we built — End-to-end math solver running in the browser.

- One unified CNN — digits **and** operators
- Safe AST evaluator (no exec / eval)
- OpenCV segmentation with proximity merging

Adding a new symbol class = more training data only, no new code.

Future work — Next steps to extend the project.

- Multi-digit numbers as a single unit
- Fractions and parentheses
- More classes: =, decimal point, variables
- Larger operator dataset → reduce failure cases
- Mobile / touch input optimization

Thank You

Questions?

Matti H. V. Lehmann

David A. Spickenheuer

Jonas Schenke

Philip Nguyen

Staniya Thomas

Abhirup Sain